MyCodeSchool

Code review — manual

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 1 — 2026-10-16 — sha 5f145ef

Am rulat fiecare funcție din `vectori.h`, cu datele tale și cu încă un set de date inventat ca să strice. Compilează curat.

Problemă	Rezultat
1, 2, 3, 4, 5, 9	✅ corecte pe datele tale
6	🔴 iese din vector pe date mari (B4)
7	🟡 algoritm corect, dar `d` greșit (M1)
8	🔴 sare peste `v[0]` (B3) · varianta binară se blochează (B2)
BAC oglindă	🔴 pică pe **exemplul din enunț** (B1)

✅ Problema 9 e interclasarea de la coadă, cu doi indici și o singură parcurgere: exact mecanismul din fișa „doi indici”. La problema 5, `i + 1` ca să deosebești „neîntâlnit” (0) de „poziția 0” e o idee bună.

🔴 B1 — Oglinda: un 0 la mijloc oprește numărarea

`vectori.h:282–284`

CUM E ACUM: pe exemplul din enunț `721 72 6 108 180 1221 121 537 1000` iese `127 72 6 108 ...`. Se aștepta `801`. Pe `505 907 12` iese `nu exista`, deși `907` are oglinda `709`.

DE CE: un număr de trei cifre `abc` are oglindă de trei cifre dacă și numai dacă `c` (ultima cifră) nu e 0. Un 0 la mijloc rămâne la mijloc: `108` → `801`. Tu faci `break` la **orice** 0, așa că pentru `108` bucla se

oprește după prima cifră, cu `ct = 1`, iar `ct == 3` pică. Singura cifră care contează e `v[i] % 10`, și o verifici o singură dată, în afara buclei.

FIX: scoate `if (cif == 0) break;` din buclă și pune condiția aici:

```
if (ct == 3 && v[i] % 10 != 0 && ogl != v[i]) {
```

💡 `solutieBAC()` n-a fost rulată niciodată: `main()` cheamă doar `solutie9()`. Rulată o singură dată cu exemplul din enunț, B1 ieșea pe loc.

🔴 B2 — Căutarea binară se blochează

`vectori.h:187–194` (`solutie8alternativ`)

CUM E ACUM: cu datele tale (`x = 8`) programul nu se mai oprește. L-am oprit eu după 3 secunde.

DE CE: căutarea binară are o singură regulă: **la fiecare pas, intervalul `[ls, ld]` trebuie să scadă**. La tine: - când `v[mij] <= x`, răspunsul e `v[mij]` sau ceva **la dreapta** lui, deci `ls = mij + 1`. Tu ai scris `mij - 1`, adică dai înapoi; - când `v[mij] > x`, nu faci nimic. `ls` și `ld` rămân aceiași, `mij` iese la fel la nesfârșit.

E a treia căutare binară cu probleme la margini, după `cautareBinara.h:40` (vectori) și `varianta73.h:18` (variante-bac).

FIX:

```
if (v[mij] <= x) { ok = 1; rez = v[mij]; ls = mij + 1; }
else ld = mij - 1;
```

🔴 B3 — Căutarea liniară sare peste primul element

`vectori.h:165` (`solutie8`)

CUM E ACUM: cu `x = 1` (`v[0] = 1`) iese `nu exista.`. Se aștepta `1`.

DE CE: `i > 0` oprește bucla înainte de `i = 0`, deci `v[0]` nu e verificat niciodată. Când parcurgi invers, ultimul indice valid e 0, iar condiția care îl include e `i >= 0`. E tiparul tău cu `<` și `<=` la

granițe. Pe datele tale nu se vede, fiindcă răspunsul e la mijloc.

FIX:

```
for (int i = d - 1; i >= 0; i--) {
```

Enunțul cere însă „algoritm eficient", adică B2. Varianta liniară rămâne doar pentru comparație.

🔴 B4 — Inserarea iese din vector

`vectori.h:117` (`solutie6`)

CUM E ACUM: cu `N = 499` numere pare (maximul din enunț), AddressSanitizer raportează `stack-buffer-overflow` la `vectori.h:124` (`v[i + 1] = 2019`).

DE CE: vectorul trebuie dimensionat pentru **rezultat**, nu pentru intrare. Fiecare număr par aduce încă un element, deci pentru 499 de numere pare rezultatul are 998 de elemente. În `v[501]` nu încap. C++ nu verifică limitele: scrie peste alte variabile din memorie și programul merge mai departe, cu valori stricate.

FIX:

```
int v[1000] = { 12,17,32,83,16,63,87 };
```

🟡 Importante

- **M1** `vectori.h:140` : `d = 7` , dar ai scris 8 valori, așa că `87` nu e citit niciodată. Iese `12 17 32 16 63` . Cu `d = 8` iese corect `12 17 32 16 63 87` , deci algoritmul e bun. E vectorul supradimensionat care ascunde un `d` greșit.
- **M2** Niciun `fstream` . Toate cele 9 probleme citesc din `numere.in` / `bac.txt` , iar 3, 4 și 5 scriu în `numere.out` . E a cincea rundă în care lipsește.
- **M3** `vectori.h:59,78,95` : la 3, 4 și 5 enunțul spune „cel mult 1 000 000 de numere", care nu încap în `v[100]` . Ideea vectorului de frecvență e că **nu mai ai nevoie de** `v` . 💡 Citește fiecare număr într-o variabilă și pune-l direct în `f` .

- **M4** `manual.cpp:7`: `main()` cheamă doar `solutie9()`. Ține un apel la fiecare `solutieX()` și rulează tot înainte de commit. Așa ar fi ieșit B1 și B2.
- **M5** `vectori.h:34`: la problema 2, cu `N = 0` nu se afișează nimic, deși răspunsul e `0`. 💡 Ce bucla rulează cel puțin o dată, chiar dacă `n` e 0?

🟢 Cleanups

- **C1** `vectori.h:162,182`: `int x = d;`. X e valoarea căutată, nu are legătură cu `d`. Doar coincid.
- **C2** `ok` primește pe rând `1` / `true` și `0` / `false` (`:184`, `:211`, `:219`, `:243`), iar `rez = -1` din `:163` ține loc de steag. Folosește `bool` cu `true` / `false` peste tot.

Before / After (doar liniile)

	Before	After
B1	`if (cif == 0) { break; }` + `ct == 3 && ogl != v[i]`	fără `break`; `ct == 3 && v[i] % 10 != 0 && ogl != v[i]`
B2	`ls = mij - 1;` și fără `else`	`ls = mij + 1;` + `else ld = mij - 1;`
B3	`i > 0`	`i >= 0`
B4	`int v[501]`	`int v[1000]`

Întrebări

1. Pentru `x = 0` (mai mic decât toate valorile), ce valori iau `ls` și `ld` la fiecare pas în B2 reparat și cu ce mesaj se termină?
2. De ce `120` nu are oglindă, iar `102` are? Ce cifră decide?
3. La problema 3: dacă nu mai păstrezi `v`, de câtă memorie ai nevoie pentru 1 000 000 de numere de cel mult 2 cifre?